- **Definições**:

```
soma_prog_f <- function(n, a1, r){
   s <- 0
   for (i in 1:n) s <- s + prog(i, a1, r)
   return(s)}

soma_prog_w <- function(n, a1, r){
   s <- 0
   while (n > 0){
      s <- s + prog(n, a1, r)
      n <- n - 1}
   return(s)}

soma_prog_r <- function(n, a1, r){
   if (n == 0) 0
   else prog(n, a1, r) + soma_prog_r(n - 1, a1, r)}
```

Exercício 56 *Na escola, aprendemos que para obter o MDC entre dois números devemos fatorá-los, sendo o MDC o produto dos fatores primos comuns elevados aos menores expoentes obtidos na fatoração. Esse método aplica diretamente o conceito de MDC (que era o que sua professora queria!). Entretando, se você bem se recorda, você nunca foi solicitado a obter o MDC de dois números como 63.954 e 24.255, por exemplo. Por quê? Porque a complexidade do algoritmo de fatoração cresce exponencialmente com o tamanho dos números tornando-o uma solução muito custosa, exceto para números pequenos.*

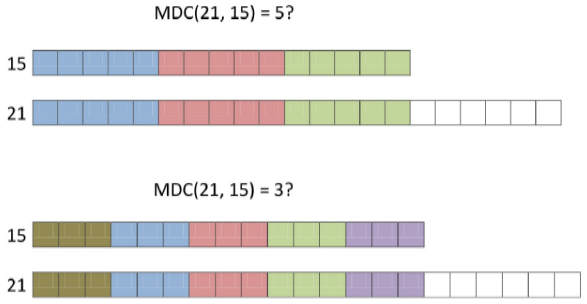


Figura 10: Esquema ilustrativo para a compreensão do algoritmo de Euclides para o cálculo do MDC.

Entretanto, Euclides percebeu que um número que pretenda ser divisor de ambos os números, deve dividir o menor deles e o que "sobra" do maior em relação ao menor. Na Figura 10, pode-se notar que 5 *não pode ser o MDC de* 15 *e* 21 *porque, se ele divide 15, ele precisaria dividir também* $(21 - 15) = 6$*, e não divide! No esquema de baixo, vê-se que* 3 *divide* 15 *e também divide a "sobra" 6. Daí, Euclides concluiu que o MDC entre dois números é também o MDC entre o menor dos números e a diferença entre o maior e o menor. Aplicado em apenas uma etapa, esse procedimento não necessariamente chegará na resposta mas certamente substitui um problema mais complexo por outro mais simples, que pode ser repetido até que o MDC seja encontrado.*

a. *Experimente no papel o algoritmo de Euclides com os números 102 e 114 e descubra a regra de parada e o MDC.*

5 Tópicos Especiais

5.1 Método da Bissecção para Aproximação de Raízes de Funções

O método da bissecção foi abordado no Exercício **??** para um número fixo n de bissecções de um intervalo inicial (a, b) contendo uma raiz real da função considerada. A questão agora é determinar n para garantir que a aproximação esteja dentro de uma tolerância fixada tol.

Segundo (RUGGIERO; LOPES,), existem ao menos duas formas de se definir a proximidade entre a estimativa para a raiz, x_n e o valor exato da raiz, r:

- considerar $|b_n - a_n| < tol$ ou
- $|f(x_n) - f(r)| = |f(x_n) - 0| = |f(x_n)| < tol$.

Será utilizado o primeiro caso, que tem solução analítica.

Sejam a_n e b_n os limites do intervalo que contém a raiz da função após a n-ésima bissecção feita no intervalo inicial (a, b). Então,

$$b_n - a_n = \frac{b_{n-1} - a_{n-1}}{2} = \frac{b_{n-2} - a_{n-2}}{4} \ldots = \frac{b-a}{2^n} < tol$$

Daí, tem-se

$$b_n - a_n = \frac{b-a}{2^n} < tol \iff 2^n < \frac{b-a}{tol}$$

Assim, n é o menor inteiro que satisfaz

$$n > log_2(b-a) - log_2(tol)$$

No R, o menor inteiro maior que um dado número é obtido com a função `ceiling()`.

Exercício 62 *Usando composição de funções, escreva a função* `bisseccao` *que recebe um vetor com os limites do intervalo que contém uma raiz real da função definida por* `f` *e a tolerância, calcula n, o número de bissecções necessárias para atingir a tolerância desejada, escreve o valor de n na tela e retorna uma lista contendo: a estimativa para a raiz e o intervalo final.*

Usando-se as funções `f`, `tem_raiz`, `subintervalo_com _raiz` *e* `bisseccao_n` *anteriormente definidas, tem-se:*

- **Nome da função***:* `bisseccao`*: vetor número* $\rightarrow$ *lista*
- **Objetivo***: Calcula aproximação para a raiz da função* `f` *pelo método da bissecção a partir de um vetor contendo o intervalo com limites* `a` *e* `b` *onde há uma raiz real, com tolerância* `tol`*.*
- **Exemplo***:* `bisseccao(c(1, 3), 0.05)` *com* $f(x) = x + exp(x)$ *retorna .*
- **Definição***:*

```
bisseccao <- function(vet, tol){
   n <- ceiling(log(b - a, 2) - log(tol, 2))
   print(n)
   s <- bisseccao_n(vet, n)
   return(list(sum(s)/2, s)}
```

Exercício 63 *Aplique* `bisseccao` *às funções* $f(x)$ *dos exercícios 42 e 43 recebendo os intervalos* (a, b) *que você identificou como tendo raízes.*

Exercício 64 *Escreva duas funções iterativas e uma recursiva para calcular f_I. Escreva funções que tenham a, h e n como parâmetro.*

- **Nome da função**: `fI`: *número número número → número*
- **Objetivo**: *Calcula aproximação por falta para a área sob uma função $f(x)$ genérica no intervalo (a, b), com base na área de* `n` *retângulos de base h.*
- **Exemplo**: `fI(5) = 0,24` *com $f(x) = x^2$*
- **Definições**:

```
fI <- function(a, h, n){
   s <- 0
   for (i in 1:n) s <- s + min(f(a + (i - 1)*h), f(a + i*h))
   return(s)}

fI <- function(a, h, n){
   s <- 0
   while(n > 0){
      s <- s + min(f(a + (n - 1)*h), f(a + n*h))
      n <- n - 1}
   return(s)}

fI <- function(a, h, n){
   if (n == 0) 0
   else min(f(a + (n - 1)*h), f(a + n*h)) + fI(a, h, n - 1)}
```

As versões de fS são obtidas substituindo-se `min` *por* `max`.

Exercício 65 *Escreva a função* `area`*, que tem como parâmetros a, b, tol e n e retorna a área sob $f(x)$ no intervalo (a, b) com base em n retângulos caso a diferença entre as aproximações por excesso e por falta sejam menores que* `tol` *ou faz nova tentativa com $n + 1$ retângulos.*

Exercício 66 *Outra abordagem consiste em aproximar a área sob a função $f(x)$ no intervalo (a, b) pela soma de áreas de trapézios. O critério de parada aqui é a diferença entre duas aproximações sucessivas, isto é, uma com n e outra com $n + 1$ trapézios. Usando controle de repetição ou recursão, escreva a função auxiliar* `area_trapezio` *tendo como parâmetros a, h e n.*

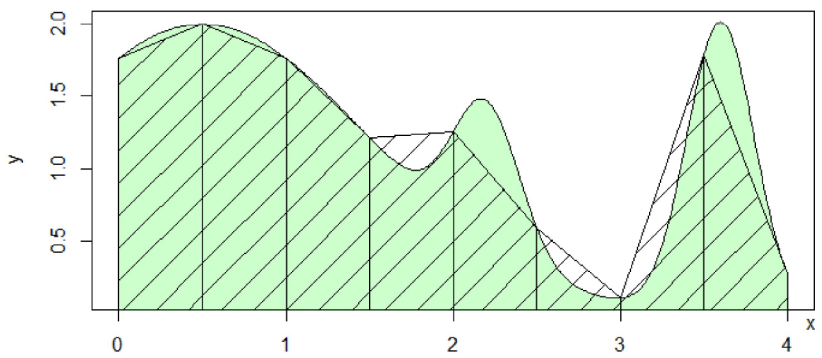


Figura 14: Aproximação pela soma de áreas de trapézios com $n = 8$.